



HACKTHEBOX



Zombienator

1st November 2023 / Document No. D23.102.123

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: Medium

Classification: Official

Synopsis

- Zombienator is a medium difficulty challenge that features making 9 allocations and 8 frees to leak a `libc` address from `tcache`, abuse `scanf("ld")` to bypass the canary check, use `pwntools struct` to pack doubles, and perform a `ret2libc` attack with one gadget. In the end, send `cat flag*>&0` to bypass `fclose(stderr)` and `fclose(stdout)`.

Description

- Our radar has detected an approaching swarm of zombies, and the threat level is high. To safeguard our community, we must mobilize an army of Zombienators to fend off this impending attack.

Skills Required

- Basic heap, `scanf` internals.

Skills Learned

- Bypass `canary` when `scanf` takes as formatter `%ld`, pack `doubles` with `pwntools`.

Enumeration

First of all, we start with a `checksec`:

```
pwndbg> checksec
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
RUNPATH:   b'./glibc/'
```

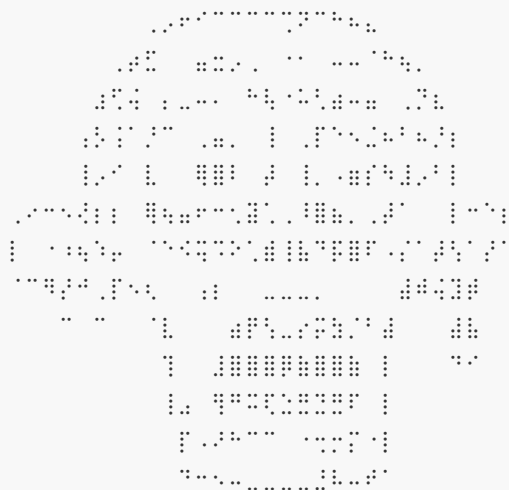
Protections

As we can see:

Protection	Enabled	Usage
Canary	✔	Prevents Buffer Overflows
NX	✔	Disables code execution on stack
PIE	✔	Randomizes the base address of the binary
RelRO	Full	Makes some binary sections read-only

As we can see, all protections are enabled.

The program's interface



```
+-----+
| Threat Level: HIGH |
+-----+
```

```
#####
#                                     #
# 1. Create  Zombienator #
# 2. Remove  Zombienator #
# 3. Display Zombienator #
# 4. Attack           #
# 5. Exit             #
#                                     #
#####

>>
```

It looks like a heap challenge. Let's head into the disassembler to get a better understanding.

Disassembly

Starting with `main()`:

```
void main(void)
{
    ulong uVar1;

    banner();
    while( true ) {
        while( true ) {
            while( true ) {
                printf(
                    "\n#####\n#                                     #\n# 1. Create
Zombienator #\n# 2. Remove  Zombienator #\n# 3. Display Zombienator #\n# 4. Attack
#\n# 5. Exit             #\n#                                     #\n#####\n\n>
> "
                );
                uVar1 = read_num();
                if (uVar1 != 4) break;
                attack();
            }
            if (4 < uVar1) goto LAB_00101a2a;
            if (uVar1 != 3) break;
            display();
        }
        if (3 < uVar1) break;
        if (uVar1 == 1) {
            create();
        }
        else {
            if (uVar1 != 2) break;
            removez();
        }
    }
}
LAB_00101a2a:
```

```
puts("\nGood luck!\n");
/* WARNING: Subroutine does not return */
exit(0x520);
}
```

As we can see there are 4 interesting function calls.

- `create()`
- `removez()`
- `display()`
- `attack()`

Taking a look at `create()`:

```
void create(void)
{
    long lVar1;
    undefined8 *puVar2;
    ulong __size;
    ulong uVar3;
    void *pvVar4;
    long in_FS_OFFSET;

    lVar1 = *(long *)(in_FS_OFFSET + 0x28);
    printf("\nZombienator\'s tier: ");
    __size = read_num();
    if ((__size < 0x83) && (__size != 0)) {
        printf("\nFront line (0-4) or Back line (5-9): ");
        uVar3 = read_num();
        if (uVar3 < 10) {
            pvVar4 = malloc(__size);
            *(void **)(z + uVar3 * 8) = pvVar4;
            puVar2 = *(undefined8 **)(z + uVar3 * 8);
            *puVar2 = 0x616e6569626d6f5a;
            puVar2[1] = 0x6461657220726f74;
            *(undefined2 *)(puVar2 + 2) = 0x2179;
            *(undefined *)((long)puVar2 + 0x12) = 0;
            printf("\n%s[+] Zombienator created!%s\n", &DAT_0010203f, &DAT_00102008);
        }
        else {
            error("[-] Invalid position!");
        }
    }
    else {
        error("[-] Cannot create Zombienator for this tier!");
    }
    if (lVar1 != *(long *)(in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
}
```

```

}
return;
}

```

The important stuff here is that we can make some allocations with size from 1-130:

- We have control over the size of `malloc` even though its limited.

```

__size = read_num();
<SNIP>
pvVar4 = malloc(__size);

```

- We also have control over the index of the allocation.

Now in `removez`:

```

void removez(void)
{
    long lVar1;
    ulong uVar2;
    long in_FS_OFFSET;

    lVar1 = *(long *)(in_FS_OFFSET + 0x28);
    printf("\nZombienator's position: ");
    uVar2 = read_num();
    if (uVar2 < 10) {
        if (*(long *)(z + uVar2 * 8) == 0) {
            error("[-] There is no Zombienator here!");
        }
        else {
            free(*(void **)(z + uVar2 * 8));
            printf("\n%s[+] Zombienator destroyed!%s\n", &DAT_0010203f, &DAT_00102008);
        }
    }
    else {
        error("[-] Invalid position!");
    }
    if (lVar1 != *(long *)(in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
    return;
}

```

After freeing the chunks, it does not NULL them, thus making it easy to leak addresses. We will make 9 allocations and then free 8 of them to fill `tcache` and place a `libc` address in the chunk. With `display()`, we will leak the `libc` address and calculate `libc` base.

```

void display(void)

```

```

{
    long lVar1;
    long in_FS_OFFSET;
    ulong local_18;

    lVar1 = *(long *)(in_FS_OFFSET + 0x28);
    putchar(10);
    for (local_18 = 0; local_18 < 10; local_18 = local_18 + 1) {
        if (*(long *)(z + local_18 * 8) == 0) {
            fprintf(stdout, "Slot [%d]: Empty\n", local_18);
        }
        else {
            fprintf(stdout, "Slot [%d]: %s\n", local_18, *(undefined8 *)(z + local_18 * 8));
        }
    }
    putchar(10);
    if (lVar1 != *(long *)(in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
    return;
}

```

Now that we have a `libc` leak from these functions, we need to escalate our exploit.

scanf("%ld")

Taking a look at `attack()`:

```

void attack(void)
{
    long in_FS_OFFSET;
    char local_121;
    ulong local_120;
    undefined local_118 [264];
    long local_10;

    local_10 = *(long *)(in_FS_OFFSET + 0x28);
    printf("\nNumber of attacks: ");
    __isoc99_scanf(&DAT_00102607, &local_121);
    for (local_120 = 0; local_120 < (ulong)(long)local_121; local_120 = local_120 + 1) {
        printf("\nEnter coordinates: ");
        __isoc99_scanf(&DAT_00102621, local_118 + local_120 * 8);
    }
    fclose(stderr);
    fclose(stdout);
    if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
}

```

```
return;  
}
```

From the manual of `scanf`:

- | | |
|---|---|
| f | Matches an optionally signed floating-point number; the next pointer must be a pointer to float . |
| l | Indicates either that the conversion will be one of d, i, o, u, x, X, or n and the next pointer is a pointer to a long int or unsigned long int (rather than int), or that the conversion will be one of e, f, or g and the next pointer is a pointer to double (rather than float). Specifying two l characters is equivalent to L. If used with %c or %s , the corresponding parameter is considered as a pointer to a wide character or wide-character string respectively. |

So, it waits for an `optionally signed long double number`. This is where the bug occurs. When `scanf("%lf")` reads symbols like `+`, `-` etc. it does not write anything on the stack, meaning we can bypass `canary` without tampering with its value. As we can see, the program kindly asks us to enter the number of attacks, while the buffer it stores the coordinates is; wrongly assumed, 264 bytes, thus letting us trigger a buffer overflow. After that, we need to store some `double` values inside this buffer.

Exploitation path

- Make 9 allocations and free 8 chunks to leak a `libc` address via `tcache`.
- Demand a big amount of attacks, at least a bit bigger than the `double` buffer that we write to.
- Send `-` or `+` or `.` to bypass overwriting the canary.
- Perform a `ret2libc` attack with `one_gadget`, making sure we satisfy all the conditions (the only one not satisfied here for the first gadget is `rbp == NULL`)
- Find a `pop rbp ; ret` gadget inside `libc` to nullify `rbp`.
- Send the payload with `pwntools repr(struct.unpack("d", p64(payload)[0]))`.
- Send `cat flag*>&0` to bypass `fclose(stderr)` and `fclose(stdout)`.